

TEAM REVIEW EDITION

WS-CON-001: Workstream Contribution Record and Compensation Boundary Specification

A locked implementation-handoff specification prepared for structured team review.

DOCUMENT	WS-CON-001
STATUS	Locked for implementation handoff
MATURITY	Design complete; not yet runtime-proven
VERSION / DATE	0.1 / 2026-07-10
OWNER	Flow Research / Workstream Engineering
SCOPE	Immutable contribution recognition, project-scoped compensation policy, frozen compensation terms, compensation awards, transactional events, external fulfillment adapters, fulfillment receipts, and contribution read views

Review instruction: validate domain decisions, invariants, transaction boundaries, failure behaviour, and conformance coverage. Proposed changes should identify the exact section and affected invariant.

Contents

Use the PDF outline or the section index below to navigate the specification.

1. Purpose	7
2. Implementation Status and Proof Boundary	7
2.1 Starting preconditions	8
2.2 Narrow precedence over WS-REV-001	8
3. Normative Language	9
4. Locked v0.1 Decisions	9
5. Actor and Authority Model	10
5.1 Relevant actors	10
5.2 Identity boundary	10
5.3 Authority matrix	11
6. Core Object Model	11
6.1 ContributionRecord	11
ContributionRecord invariants	12
6.2 CompensationPolicy	12
6.3 CompensationPolicyVersion	13
Version invariants	13
6.4 CompensationRule	13
Rule invariants	13
6.5 CompensationAwardDefinition	13
Definition invariants	14
6.6 Frozen compensation references	14
6.7 CompensationAward	15
CompensationAward invariants	15
6.8 ProjectCompensationAdapterBinding	15
Binding invariants	15
6.9 CompensationFulfillmentReceipt	16
Receipt invariants	16
6.10 CompensationStatusProjection	16
7. Project Scope and Global Contribution Views	17
7.1 Global record identity	17
7.2 Project-scoped economics	17
7.3 Read views	17
8. Compensation Policy Lifecycle	18
8.1 Create draft	18
8.2 Validate draft	18

8.3 Publish and activate	18
8.4 Explicit unpaid policy	18
8.5 Policy-change boundary	19
9. Contribution Creation Matrix	19
10. State Machines	19
10.1 ContributionRecord lifecycle	19
10.2 CompensationPolicyVersion	20
10.3 CompensationAward fulfillment projection	20
11. Frozen-Terms Operations	20
11.1 Create TaskAssignment	20
11.2 Create ReviewLease	20
11.3 Released, expired, or revoked lease	21
12. Atomic Review-to-Contribution Operations	21
12.1 Common decision transaction	21
12.2 Reviewer contribution creation	21
12.3 Submitter contribution creation on accept	22
12.4 Needs revision	22
12.5 Reject	22
12.6 Explicitly unpaid result	23
12.7 Integrity failure	23
13. Compensation Evaluation	23
13.1 Deterministic algorithm	23
13.2 Decision neutrality	23
13.3 Multiple instruments	24
13.4 Decimal and unit rules	24
14. Transactional Event Contract	24
14.1 Outbox requirement	24
14.2 Common event envelope	24
14.3 ContributionRecorded	25
14.4 CompensationAwardCreated	25
14.5 CompensationFulfillmentRequested	25
14.6 CompensationFulfillmentRecorded	26
14.7 Sensitive-data restriction	26
15. External Adapter Contract	27
15.1 Workstream adapter port	27
15.2 Delivery	27
15.3 Adapter idempotency	27
15.4 Fulfillment callback request	27
15.5 Callback authentication and authorization	28

15.6 Fulfilled callback	28
15.7 Failed callback	28
15.8 Prohibited partial fulfillment	29
15.9 Contradictory callbacks	29
16. Reputation Boundary	29
17. Read Model and API Composition	30
17.1 Canonical versus composed data	30
17.2 Example contribution response	30
18. API Contract	30
18.1 Compensation policies	30
18.2 Adapter bindings	31
18.3 Contributions	32
18.4 Compensation awards	32
18.5 Fulfillment callback	33
19. Error Contract	33
20. Database Constraints and Indexes	34
21. Concurrency and Idempotency	35
21.1 Review-decision replay	35
21.2 Concurrent decision attempts	35
21.3 Concurrent policy activation	35
21.4 Policy activation versus assignment	36
21.5 Award duplication	36
21.6 Outbox delivery replay	36
21.7 Concurrent callbacks	36
21.8 Callback before delivery acknowledgement	36
22. Background Processing and Recovery	37
22.1 Outbox dispatcher	37
22.2 Status projection updater	37
22.3 Reconciliation job	37
22.4 Event replay	38
22.5 Adapter suspension	38
23. Audit Events	38
Policy	38
Frozen terms	38
Contribution	39
Award and delivery	39
Callback	39

24. Notifications and Operational Views	39
24.1 Contributor view	40
24.2 Project Admin view	40
24.3 Finance Authority view	40
24.4 Reputation Authority view	40
25. Observability	41
26. Security Requirements	41
27. End-to-End Reference Sequences	42
27.1 Accepted submission with paid reviewer and paid submitter	42
27.2 Needs revision with reviewer compensation	42
27.3 Reject with reviewer compensation	43
27.4 Explicitly unpaid project	43
27.5 Money plus project points	43
27.6 Adapter failure followed by success	43
27.7 Policy changes during existing work	43
27.8 Duplicate decision and duplicate callback	44
28. Conformance Tests	44
28.1 Policy tests	44
28.2 Freeze tests	44
28.3 Contribution tests	45
28.4 Atomicity and idempotency tests	45
28.5 Compensation evaluation tests	45
28.6 Delivery tests	46
28.7 Callback tests	46
28.8 Read and recovery tests	46
29. Implementation Delivery Order	47
Phase 1: Schema and immutable policy model	47
Phase 2: Policy and binding APIs	47
Phase 3: Freeze integration	47
Phase 4: Atomic Review integration	48
Phase 5: External adapter boundary	48
Phase 6: Read APIs and operations	48
Phase 7: Live API drill	48
30. Definition of Done	49
31. Explicitly Out of Scope	49
32. Future Additive Directions	50
33. Coding-Agent Handoff Rules	51

34. Final Implementation Contract

51

1. Purpose

This specification defines how Workstream recognizes completed contributor work after a valid human review decision.

It establishes:

- when a submitter has produced a recognized contribution;
- when a reviewer has produced a recognized contribution;
- how contribution recognition remains independent of the review outcome;
- how a global contribution history remains filterable by project;
- how project compensation terms are versioned and frozen before work begins;
- how money and project-points awards are derived deterministically;
- how Workstream hands fulfillment to external adapters;
- how adapter acknowledgements differ from fulfillment results;
- how Workstream records fulfillment results without mutating contribution history;
- how duplicate events, callbacks, retries, failures, and races are handled;
- which responsibilities stop at the Workstream boundary.

The fundamental model is:

```
Valid Review recorded
|
+--> Reviewer ContributionRecord
|   +--> ContributionRecord
|   |   +--> zero or more CompensationAwards
|   |
|   +--> if decision = accept:
|       |   Submitter ContributionRecord
|       |   |   +--> ContributionRecord
|       |   |   +--> zero or more CompensationAwards
|       |
|       +--> zero or more CompensationAwards

Each CompensationAward
|
+--> CompensationFulfillmentRequested
|   |
|   +--> external money or project-points adapter
|       |
|       +--> CompensationFulfillmentReported
|           |
|           +--> immutable fulfillment receipt
|           +--> mutable status projection
```

Workstream records the fact that work was performed and the exact project-scoped compensation authorized for it. Workstream does not implement payment-provider transactions, point balances, or provider ledgers.

2. Implementation Status and Proof Boundary

The coding agent **MUST** preserve the difference between proven behaviour, locked design, and deferred work.

Lifecycle area	Status
Project guide, submission policy, task lock, submission finalization, and durable checker run	Proven
Transition through review_pending	Proven

Lifecycle area	Status
Reviewer routing, leases, immutable decisions, and revision chain	Locked by WS-REV-001; not yet runtime-proven
Reviewer Contribution Record for every completed Review	Locked by this specification
Submitter Contribution Record after accept	Locked by this specification
Project CompensationPolicy and immutable versions	Locked by this specification
Money and project-points CompensationAwards	Locked by this specification
External fulfillment event and callback boundary	Locked by this specification
Provider payment requests, attempts, settlement, balances, and reconciliation	External and deferred
Project-points ledger and balances	External and deferred
Reputation scoring formula and aggregate	Deferred
Adjudication and contribution adjustment	Deferred

2.1 Starting preconditions

This specification begins when WS-REV-001 is recording a valid Review against an active ReviewLease.

The Review decision may be:

- accept;
- needs_revision;
- reject.

No Contribution Record may be created for:

- an automated checker result;
- a pending or merely claimed review;
- an expired lease;
- a manually released lease;
- an administratively revoked lease;
- an invalid or rolled-back Review transaction.

2.2 Narrow precedence over WS-REV-001

WS-REV-001 section 12.1 uses ContributionRecordRequested as a temporary integration event because contribution creation was deferred to the next specification.

After WS-CON-001 is implemented, the following rules supersede only that temporary seam:

1. Every valid Review transaction creates the reviewer Contribution Record directly.
2. An accept Review transaction additionally creates the submitter Contribution Record directly.
3. Those records, their CompensationAwards, and their outbox events are committed atomically with the Review.
4. ContributionRecordRequested MUST NOT be emitted by the completed implementation.
5. ContributionRecorded and CompensationFulfillmentRequested are the canonical downstream events.

All other WS-REV-001 authority, routing, lease, revision, reject, and immutability rules remain unchanged.

3. Normative Language

The terms MUST, MUST NOT, REQUIRED, SHOULD, SHOULD NOT, and MAY are normative.

- MUST or REQUIRED means the implementation is non-conformant if the rule is absent.
- MUST NOT means the behaviour is prohibited.
- SHOULD means deviation requires an explicit Architecture Decision Record.
- MAY means the implementation choice is allowed without changing domain behaviour.

Database time is authoritative for all transaction timestamps. Decimal quantities MUST never be represented or calculated using binary floating-point types.

Version notation: Workstream and this lifecycle specification are release v0.1. The /v1 API prefix, event_version: 1, event idempotency suffix :v1, route-key suffix -v1, and references such as SubmissionVersion(v1) are independent API, event-schema, route, or submission-version identifiers. None denotes a Workstream v1.0 release.

4. Locked v0.1 Decisions

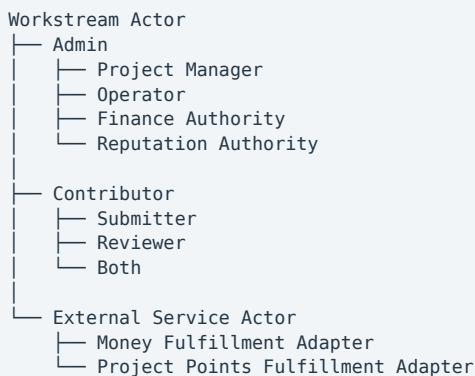
The following decisions are not left to coding-agent interpretation.

1. Contribution Records are globally addressable in Workstream and always carry a mandatory project_id.
2. Global contribution history can be filtered by project, contributor, task, contribution type, and time.
3. A valid completed Review always creates exactly one reviewer Contribution Record, regardless of accept, needs_revision, or reject.
4. An accept Review additionally creates exactly one submitter Contribution Record.
5. Needs-revision and reject Reviews do not create submitter Contribution Records.
6. Every completed revision-review round is an independently recognized reviewer contribution.
7. Expired, released, and revoked review leases create no contribution and no compensation.
8. Contribution Records are immutable and have no mutable fulfillment fields.
9. Review, contribution, award, and outbox creation occur in one database transaction.
10. Every project MUST bind an active published CompensationPolicyVersion before new TaskAssignments or ReviewLeases can be created.
11. Unpaid work uses an explicit policy rule with no awards; missing policy configuration is never treated as unpaid.
12. V0.1 compensation is fixed per project and contribution type.
13. V0.1 has no contributor-specific rates, manually negotiated rates, compensation tiers, or task-specific overrides.
14. Submitter compensation terms are frozen on TaskAssignment creation.
15. Reviewer compensation terms are frozen on successful ReviewLease creation.
16. A later project-policy change never alters an existing TaskAssignment, ReviewLease, Contribution Record, or CompensationAward.
17. V0.1 supports only money and project_points compensation instruments.
18. Monetary awards are project obligations even when their unit is a standard currency.
19. Project points belong to a project-specific namespace and are not global Workstream points.

20. A contribution may create no award, one award, or two awards: at most one money award and at most one project-points award.
21. Workstream creates immutable CompensationAwards and emits fulfillment events.
22. External adapters own payment requests, provider attempts, finance approval, point ledgers, balances, provider retries, and provider reconciliation.
23. Workstream retries event delivery only until the external adapter acknowledges receipt.
24. Adapter delivery acknowledgement is not proof of compensation fulfillment.
25. Fulfillment is reported later through an authenticated, idempotent callback.
26. V0.1 fulfillment outcomes are fulfilled or failed; partial fulfillment is prohibited.
27. A failed award may later receive a valid fulfilled receipt.
28. A fulfilled award is terminal and cannot later become failed.
29. Fulfillment responses create immutable receipts and update a derived status projection; they never mutate the Contribution Record or CompensationAward.
30. Reputation consumes ContributionRecorded independently of compensation fulfillment.
31. Reputation scoring formulas and reputation aggregates are not implemented by this specification.
32. Acceptance is final in v0.1; no adjudication or adjustment lifecycle exists.
33. All provider-specific and ledger-specific objects remain outside Workstream.

5. Actor and Authority Model

5.1 Relevant actors



External adapters are service actors. They are not Admins or Contributors and receive no authority over Reviews, tasks, assignments, policies, or Contribution Records.

5.2 Identity boundary

All human and service requests pass through the Identity Issuer boundary.

```

Identity Issuer token
-> Workstream verifies issuer, subject, audience, expiry, and scopes
-> Workstream resolves local actor or adapter registration
-> Workstream checks project and instrument scope
-> Workstream authorizes the exact operation
  
```

A valid service token is insufficient by itself. The service actor must also match the adapter binding frozen on the referenced CompensationAward.

5.3 Authority matrix

Action	Required authority
Create CompensationPolicy draft or version draft	Project Manager, Finance Authority, or Operator
Publish or activate a policy containing money	Finance Authority or Operator
Publish or activate a project-points-only or explicitly unpaid policy	Project Manager or Operator
Retire an active money policy	Finance Authority or Operator
Retire a points-only or unpaid policy	Project Manager or Operator
Configure money-adapter binding	Finance Authority or Operator
Configure project-points adapter binding	Project Manager or Operator
Read own Contribution Records	Contributor owning the records
Read project Contribution Records	Project Manager, Operator, Finance Authority, or Reputation Authority within authorized project scope
Read global cross-project contribution history	Operator, or contributor reading only their own history
Create Contribution Record	System transaction only
Create CompensationAward	System transaction only
Report fulfillment	Bound external adapter for the exact award instrument and project

No API permits an Admin, Contributor, or adapter to create or edit a Contribution Record directly.

6. Core Object Model

6.1 ContributionRecord

ContributionRecord is the permanent recognition that one contributor completed one eligible unit of work.

```

ContributionRecord:
  id: uuid
  project_id: uuid
  task_id: uuid
  contributor_id: uuid

  contribution_type: enum [
    accepted_submission,
    completed_review
  ]

  source_review_id: uuid
  source_review_lease_id: uuid | null
  source_task_assignment_id: uuid
  submission_version_id: uuid

  compensation_policy_version_id: uuid

  artifact_hash: string
  recorded_at: timestamp
  recorded_by: system_actor_id

  correlation_id: string
  causation_event_id: uuid | null

```

ContributionRecord invariants

- The record is immutable from insertion.
- `project_id`, `task_id`, `contributor_id`, `source` `Review`, `assignment`, `submission` `version`, and `compensation` `version` **MUST** all belong to one internally consistent chain.
- `contribution_type = accepted_submission` requires `Review.decision = accept`.
- `contribution_type = accepted_submission` requires `contributor_id` to equal the `submission` `creator` and `TaskAssignment` `contributor`.
- `contribution_type = completed_review` requires `contributor_id` to equal `ReviewReviewer.id`.
- `contribution_type = completed_review` requires `source_review_lease_id` to equal `Review.review_lease_id`.
- `accepted_submission` records reference the active submitter `TaskAssignment` completed by the `accept` `Review`.
- `completed_review` records reference the `TaskAssignment` whose work was reviewed but do not attribute that assignment to the reviewer.
- `artifact_hash` equals the immutable `SubmissionVersion` `artifact` `hash`.
- `compensation_policy_version_id` comes from the frozen `TaskAssignment` field for `accepted_submission`.
- `compensation_policy_version_id` comes from the frozen `ReviewLease` field for `completed_review`.
- Exactly one `accepted_submission` record may exist per `accepted` `Review`.
- Exactly one `completed_review` record may exist per `Review`.
- An `accept` `Review` therefore creates exactly two `Contribution` `Records` unless one of the contributors is represented by a prohibited self-review, which WS-REV-001 prevents.
- Needs-revision and reject `Reviews` create exactly one `Contribution` `Record`: `completed_review`.
- A record may exist even when its `compensation` `policy` produces no awards.
- The record has no `payment` `status`, `points` `status`, `mutable` `status`, `deleted` `at`, or `editable` `metadata`.

Recommended uniqueness constraints:

```
UNIQUE(source_review_id, contribution_type)
UNIQUE(id, project id)
```

6.2 CompensationPolicy

CompensationPolicy is the stable project-owned policy identity.

```
CompensationPolicy:
  id: uuid
  project_id: uuid
  name: string
  status: enum [draft, active, retired]
  current_published_version_id: uuid | null

  created_by: actor_id
  created_at: timestamp
  retired_by: actor_id | null
  retired_at: timestamp | null
```

One project may have multiple historical policy identities, but at most one `CompensationPolicy` and, when the project is configured for new work, exactly one published version are active at a time. A newly created policy remains draft with a null current published version id until its first version is published.

6.3 CompensationPolicyVersion

```
CompensationPolicyVersion:
  id: uuid
  compensation_policy_id: uuid
  project_id: uuid
  version_number: int
  status: enum [draft, published, retired]

  created_by: actor_id
  created_at: timestamp
  published_by: actor_id | null
  published_at: timestamp | null
  retired_by: actor_id | null
  retired_at: timestamp | null
```

Version invariants

- version_number starts at 1 and increases monotonically within one CompensationPolicy.
- Draft versions may be edited only through explicit draft APIs.
- Publishing validates the complete version and makes every field and award definition immutable.
- A published version may be retired but never edited or deleted.
- Retiring a version does not invalidate frozen references from assignments or leases.
- A retired version remains valid for contribution and award creation when it was frozen before retirement.
- At most one published version is active for new work within a project.

6.4 CompensationRule

Every policy version contains exactly one rule for each contribution type. This makes unpaid work explicit rather than indistinguishable from missing configuration.

```
CompensationRule:
  id: uuid
  compensation_policy_version_id: uuid
  project_id: uuid
  contribution_type: enum [
    accepted_submission,
    completed_review
  ]
  compensation_mode: enum [compensated, unpaid]
```

Rule invariants

- Exactly one accepted_submission rule and one completed_review rule exist in every publishable version.
- A compensated rule has at least one and at most two award definitions.
- An unpaid rule has zero award definitions.
- Rule and definitions become immutable together when the version is published.
- V0.1 rules have no condition expression, priority, fallback, multiplier, or outcome selector.

6.5 CompensationAwardDefinition

Each published version defines fixed awards for the two contribution types.

```

CompensationAwardDefinition:
  id: uuid
  compensation_rule_id: uuid
  compensation_policy_version_id: uuid
  project_id: uuid

  contribution_type: enum [
    accepted_submission,
    completed_review
  ]

  instrument_type: enum [
    money,
    project_points
  ]

  unit_code: string
  quantity: decimal
  adapter_binding_id: uuid

```

Definition invariants

- At most one definition exists for each `contribution_type` and `instrument_type` within a version.
- `quantity` MUST be greater than zero.
- Money `unit_code` MUST be an uppercase ISO 4217 currency code configured for the project.
- Project-points `unit_code` MUST match the project-scoped point unit configured by the policy.
- A points unit has identity (`project_id`, `unit_code`). The same text in another project is a different unit.
- `adapter_binding_id` must match the same project and instrument.
- Published definitions are immutable.
- An explicitly unpaid rule has zero award definitions.
- Zero-quantity award definitions are prohibited.
- V0.1 rule evaluation is exact matching on `contribution_type` only. No other condition, multiplier, tier, score, task class, contributor class, or outcome rule is allowed.

6.6 Frozen compensation references

WS-CON-001 extends Project and two existing work objects.

```

Project:
  # existing fields omitted
  active_compensation_policy_version_id: uuid | null

```

```

TaskAssignment:
  # existing fields omitted
  submitter_compensation_policy_version_id: uuid

```

```

ReviewLease:
  # existing fields omitted
  reviewer_compensation_policy_version_id: uuid

```

The references are set once:

- TaskAssignment freezes the project active version in the assignment-creation transaction.
- ReviewLease freezes the project active version in the successful claim transaction.
- Neither field may be updated later.
- Policy retirement does not change either field.

6.7 CompensationAward

CompensationAward is Workstream's immutable statement of what the project authorized for one contribution.

```
CompensationAward:
  id: uuid
  project_id: uuid
  contribution_record_id: uuid
  contributor_id: uuid

  compensation_policy_version_id: uuid
  award_definition_id: uuid
  adapter_binding_id: uuid

  instrument_type: enum [
    money,
    project_points
  ]

  unit_code: string
  quantity: decimal

  created_at: timestamp
  correlation_id: string
```

CompensationAward invariants

- Immutable from insertion.
- Project, contributor, policy version, and contribution must match.
- Instrument, unit, quantity, and adapter binding are copied exactly from the published definition.
- At most one award exists per contribution and instrument type.
- A completed_review award does not depend on Review.decision.
- No award may be created for a released, expired, or revoked lease.
- No award may be created by an adapter callback.
- Failed or fulfilled status is never stored on this record.

Recommended uniqueness constraint:

```
UNIQUE(contribution_record_id, instrument_type)
```

6.8 ProjectCompensationAdapterBinding

The binding identifies the logical adapter authorized for one project and instrument.

```
ProjectCompensationAdapterBinding:
  id: uuid
  project_id: uuid
  instrument_type: enum [money, project_points]
  adapter_actor_id: actor_id
  route_key: string
  status: enum [active, suspended, retired]

  created_by: actor_id
  created_at: timestamp
  suspended_at: timestamp | null
  retired_at: timestamp | null
```

Binding invariants

- At most one active binding exists per project and instrument.

- Published award definitions reference a binding for the same project and instrument.
- The binding identity is frozen into each CompensationAward.
- Endpoint, credentials, and provider configuration are deployment secrets and are not domain fields.
- Suspending a binding pauses new delivery but does not cancel or alter awards.
- Retiring a binding is prohibited while it is referenced by an active policy version, by unfinished work whose frozen policy version contains it, or by an unfulfilled award.
- A service actor bound to money cannot report project-points fulfillment and vice versa.

6.9 CompensationFulfillmentReceipt

Each valid callback creates an immutable receipt.

```
CompensationFulfillmentReceipt:  
  id: uuid  
  compensation_award_id: uuid  
  project_id: uuid  
  adapter_binding_id: uuid  
  
  external_event_id: string  
  reported_status: enum [fulfilled, failed]  
  
  external_reference: string | null  
  fulfilled_quantity: decimal | null  
  fulfilled_at: timestamp | null  
  
  failure_code: string | null  
  failure_message: string | null  
  
  reported_at: timestamp  
  received_at: timestamp  
  correlation_id: string
```

Receipt invariants

- Immutable from insertion.
- external_event_id is globally unique per adapter binding.
- fulfilled requires external_reference, fulfilled_quantity, and fulfilled_at.
- fulfilled_quantity must equal the CompensationAward quantity exactly.
- failed requires failure_code.
- failed must have null fulfilled_quantity and fulfilled_at.
- Project, binding, contributor, instrument, unit, and authorized quantity cannot be changed by the callback.
- Multiple failed receipts may exist for one award when their external_event_id values differ.
- At most one fulfilled receipt may exist for one award.
- A fulfilled award rejects all later failed or fulfilled callbacks except an exact idempotent replay of the accepted fulfilled callback.

6.10 CompensationStatusProjection

This projection exists only for efficient reads and operations.

```

CompensationStatusProjection:
  compensation_award_id: uuid
  delivery_status: enum [
    pending_delivery,
    acknowledged_by_adapter
  ]
  fulfillment_status: enum [
    pending,
    failed,
    fulfilled
  ]

  latest_receipt_id: uuid | null
  external_reference: string | null
  last_failure_code: string | null
  delivered_at: timestamp | null
  fulfilled_at: timestamp | null
  updated_at: timestamp

```

The projection is mutable and rebuildable. It is not the source of truth. ContributionRecord, CompensationAward, outbox delivery history, and CompensationFulfillmentReceipt are authoritative.

7. Project Scope and Global Contribution Views

7.1 Global record identity

ContributionRecord.id is globally unique within Workstream. A contributor's complete history may span projects.

```

Contributor
├── Contribution history
│   ├── Project A
│   │   ├── accepted_submission
│   │   └── completed_review
│   ├── Project B
│   │   └── completed_review
│   └── Project C
│       └── accepted_submission

```

7.2 Project-scoped economics

Every CompensationPolicy, policy version, award definition, CompensationAward, adapter binding, and fulfillment receipt has a project_id.

- A Project A policy cannot compensate Project B work.
- Project A points cannot be fulfilled into Project B's point namespace.
- A USD award from Project A remains Project A's obligation; it is not a global Workstream balance.
- Cross-project totals are read-model calculations only.

7.3 Read views

Required views:

- contributor global contribution history;
- contributor history filtered to one project;
- project contribution history;
- task contribution history;
- Contribution Record detail with derived compensation summary;

- project outstanding compensation awards by instrument and age;
- adapter delivery and fulfillment operational view.

8. Compensation Policy Lifecycle

8.1 Create draft

An authorized Admin creates a CompensationPolicy and version 1 draft, or creates the next draft version under an existing policy.

Draft creation does not affect existing or new work until publication and activation.

8.2 Validate draft

Publication validation MUST confirm:

- policy and every definition belong to the same project;
- only accepted_submission and completed_review rules exist;
- only money and project_points instruments exist;
- no duplicate instrument exists within a contribution type;
- every quantity is a positive decimal;
- every unit code is valid for its instrument;
- every adapter binding belongs to the project, supports the instrument, and is active;
- both contribution types are explicitly present as either compensated or unpaid;
- no task, contributor, tier, outcome, skill, or reputation condition exists;
- the actor has sufficient authority for every instrument.

8.3 Publish and activate

Publication and project activation occur in one transaction:

```
validate complete draft
lock project's active compensation policy reference
publish version immutably
set owning CompensationPolicy.status -> active
set owning CompensationPolicy.current_published_version_id -> new version id
retire any different previously active CompensationPolicy identity
set project active_compensation_policy_version_id
retire prior version for new-work selection
write policy audit events
commit
```

Retiring the prior version means it is no longer selected for new assignments or leases. It remains valid for previously frozen work.

If an active version is retired without an immediate replacement, the same transaction sets Project.active_compensation_policy_version_id to null and CompensationPolicy.status to retired. New TaskAssignments and ReviewLeases then fail until another version is activated. Existing frozen work remains valid.

8.4 Explicit unpaid policy

An unpaid project still requires a published version.

```
accepted_submission:
  compensation_mode: unpaid
  awards: []

completed_review:
  compensation_mode: unpaid
  awards: []
```

This prevents missing configuration from silently removing contributor compensation.

8.5 Policy-change boundary

If version 2 becomes active:

- existing TaskAssignments retain their frozen version;
- existing ReviewLeases retain their frozen version;
- new TaskAssignments freeze version 2;
- new ReviewLeases freeze version 2;
- queued but unleased ReviewQueueEntries use the version active when a reviewer successfully claims them;
- Contribution Records always copy the version from their assignment or lease, never from the project's current setting at decision time.

9. Contribution Creation Matrix

Review outcome or lease result	Reviewer Contribution Record	Submitter Contribution Record	Reviewer compensation evaluated	Submitter compensation evaluated
accept	Yes	Yes	Yes	Yes
needs_revision	Yes	No	Yes	No
reject	Yes	No	Yes	No
manual release	No	No	No	No
lease expiry	No	No	No	No
administrative revocation	No	No	No	No
invalid decision attempt	No	No	No	No
idempotent replay of committed decision	Return existing	Return existing if accept	Return existing	Return existing if accept

Review compensation is decision-neutral. No award definition may match or vary by decision.

10. State Machines

10.1 ContributionRecord lifecycle

ContributionRecord has no mutable state machine.

```
not present --valid Review transaction--> recorded permanently
```

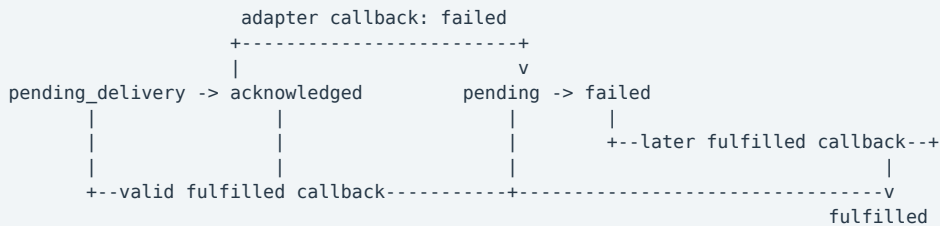
It cannot move to pending, failed, fulfilled, cancelled, voided, or deleted.

10.2 CompensationPolicyVersion

```
draft --publish and activate--> published --replace for new work--> retired
```

- draft may be edited;
- published is immutable and selectable for new work;
- retired is immutable and valid only through frozen references.

10.3 CompensationAward fulfillment projection



Normative rules:

- Delivery acknowledgement changes only delivery_status.
- A failed callback changes fulfillment_status to failed.
- A later valid fulfilled callback changes failed to fulfilled.
- Fulfilled is terminal.
- Delivery state and fulfillment state are independent.
- A callback may arrive before local delivery acknowledgement; it is accepted if otherwise valid and MUST NOT later regress.

11. Frozen-Terms Operations

11.1 Create TaskAssignment

Preconditions:

- project has an active published CompensationPolicyVersion;
- submitter has the required project role grant;
- task and project are claimable under existing rules.

Transaction:

```
lock project active compensation policy reference
validate version is published
create TaskAssignment
copy version id -> submitter_compensation_policy_version_id
write CompensationTermsFrozenForAssignment
commit
```

If no active version exists, assignment creation fails with compensation_policy_missing.

11.2 Create ReviewLease

This extends the atomic WS-REV-001 claim transaction.

Preconditions:

- every WS-REV-001 eligibility and capacity rule passes;
- project has an active published CompensationPolicyVersion.

Transaction addition:

```
lock project active compensation policy reference
validate version is published
create ReviewLease
copy version id -> reviewer_compensation_policy_version_id
write CompensationTermsFrozenForReviewLease
commit with queue claim
```

If the project changes policy concurrently, row locking ensures the lease freezes either the old active version before replacement commits or the new active version after replacement commits. It cannot freeze an indeterminate mixture.

11.3 Released, expired, or revoked lease

The frozen reference remains on the permanent ReviewLease attempt for audit. It creates no contribution or award because no valid Review was recorded.

12. Atomic Review-to-Contribution Operations

12.1 Common decision transaction

Every valid Review decision uses one PostgreSQL transaction and one unit of work.

After all WS-REV-001 validations pass, the transaction MUST:

```
1. create immutable Review and any ReviewFinding/FindingResolution records
2. consume ReviewLease and close ReviewQueueEntry
3. apply the decision-specific Task and TaskAssignment transition
4. create reviewer ContributionRecord
5. evaluate the ReviewLease's frozen policy version
6. create reviewer CompensationAwards from exact completed_review definitions
7. create reviewer ContributionRecorded outbox event
8. create CompensationAwardCreated audit/outbox event for each award
9. create CompensationFulfillmentRequested outbox event for each award
10. if decision = accept:
    a. create submitter ContributionRecord
    b. evaluate the TaskAssignment's frozen policy version
    c. create submitter CompensationAwards from exact accepted_submission definitions
    d. create submitter ContributionRecorded outbox event
    e. create award and fulfillment-requested events for each award
11. write all review, contribution, compensation, task, and assignment audit events
12. commit
```

If any database insert, invariant, event write, or decision-specific transition fails, the entire transaction rolls back.

No adapter call occurs inside this transaction. External availability therefore cannot delay or invalidate the human decision.

12.2 Reviewer contribution creation

Input:

- canonical Review being created;
- consumed ReviewLease;

- reviewed SubmissionVersion;
- reviewed TaskAssignment;
- frozen ReviewLease reviewer_compensation_policy_version_id.

Output:

```
contribution_type: completed_review
contributor_id: Review.reviewer_id
source_review_id: Review.id
source_review_lease_id: ReviewLease.id
source_task_assignment_id: TaskAssignment.id
submission_version_id: Review.submission_version_id
compensation_policy_version_id: ReviewLease.reviewer_compensation_policy_version_id
```

The algorithm is identical for accept, needs_revision, and reject. Branching compensation logic by Review.decision is prohibited.

12.3 Submitter contribution creation on accept

Input:

- Review with decision = accept;
- accepted SubmissionVersion;
- completed TaskAssignment;
- frozen TaskAssignment submitter_compensation_policy_version_id.

Output:

```
contribution_type: accepted_submission
contributor_id: SubmissionVersion.created_by
source_review_id: Review.id
source_review_lease_id: null
source_task_assignment_id: TaskAssignment.id
submission_version_id: Review.submission_version_id
compensation_policy_version_id: TaskAssignment.submitter_compensation_policy_version_id
```

The contribution is created only after all accept validations pass but before transaction commit.

12.4 Needs revision

Needs revision creates:

- one completed_review Contribution Record;
- zero or more reviewer CompensationAwards;
- no accepted_submission Contribution Record;
- no submitter CompensationAward.

When a later submission version is reviewed, that later Review creates another completed_review record. There is no maximum compensated review-round count in v0.1.

Structured findings, immutable chains, no-self-review, reviewer leasing, and audit events remain the controls against abusive revision cycling.

12.5 Reject

Reject creates:

- one completed_review Contribution Record;
- zero or more reviewer CompensationAwards;

- no submitter Contribution Record;
- no submitter CompensationAward.

Reviewer compensation remains valid even though the submitter TaskAssignment becomes blocked and the task closes.

12.6 Explicitly unpaid result

If the frozen policy version has no award definition for the relevant contribution type:

- create the Contribution Record;
- create ContributionRecorded;
- create no CompensationAward;
- create no CompensationFulfillmentRequested event;
- do not treat the absence of an award as an error.

The Contribution Record's policy version proves that the explicit unpaid rule was evaluated.

12.7 Integrity failure

If a frozen policy reference is missing, corrupt, from the wrong project, or contains prohibited duplicate definitions:

- reject the decision transaction with compensation_policy_integrity_error;
- create no Review or contribution;
- after rollback, write a structured integrity-error log, increment workstream_compensation_policy_integrity_errors_total, and trigger the configured operator alert;
- require operator remediation.

The implementation MUST NOT substitute the current project policy or silently treat the work as unpaid.

13. Compensation Evaluation

13.1 Deterministic algorithm

For one Contribution Record:

```
load exact frozen CompensationPolicyVersion
validate project match
select the one CompensationRule matching record.contribution_type
if rule is unpaid, return zero awards
if rule is compensated, load its award definitions
order definitions deterministically by instrument_type
for each definition:
  copy definition fields into immutable CompensationAward
  resolve the exact frozen adapter binding
  create award and outbox events
return created awards
```

No scoring engine, rules interpreter, dynamic expression, network call, current reputation lookup, or current project-policy lookup is permitted.

13.2 Decision neutrality

For completed_review:

```
award(completed_review, accept)
  = award(completed_review, needs_revision)
  = award(completed_review, reject)
```

The Review outcome may be included as provenance in ContributionRecorded, but it cannot select or modify compensation.

13.3 Multiple instruments

If a completed-review rule defines money and project points:

```
one ContributionRecord
  -> one money CompensationAward
  -> one project_points CompensationAward
  -> two independent fulfillment events
```

One instrument's delivery or fulfillment status never blocks or changes the other.

13.4 Decimal and unit rules

- Use an exact decimal database type.
- API quantities are decimal strings, never JSON floating-point numbers.
- Money quantities are interpreted according to unit_code but are not rounded by Workstream after policy publication.
- Policy publication rejects quantities exceeding the configured currency precision.
- Project-points quantities MUST be whole positive numbers in v0.1.
- Unit conversion is prohibited.
- Adapters cannot substitute another currency or point unit.

14. Transactional Event Contract

14.1 Outbox requirement

Every outbound domain or integration event is written to the existing transactional outbox in the same transaction as its source record.

The dispatcher uses at-least-once delivery. Consumers and adapters MUST implement idempotency.

No code path may:

- commit a Contribution Record and publish its event outside the outbox;
- publish an event before the database transaction commits;
- call an external adapter from the Review transaction;
- delete an outbox event because delivery is temporarily failing.

14.2 Common event envelope

```

event_id: uuid
event_type: string
event_version: 1
occurred_at: timestamp
producer: workstream

project_id: uuid
correlation_id: string
causation_event_id: uuid | null
idempotency_key: string

payload: object

```

The canonical idempotency key is stable for the logical event and does not change across delivery retries.

14.3 ContributionRecorded

One event is emitted per Contribution Record.

```

event_type: ContributionRecorded
event_version: 1
idempotency_key: "contribution:{contribution_record_id}:recorded:v1"

payload:
  contribution_record_id: uuid
  project_id: uuid
  task_id: uuid
  contributor_id: uuid
  contribution_type: accepted_submission | completed_review
  source_review_id: uuid
  source_review_lease_id: uuid | null
  source_task_assignment_id: uuid
  submission_version_id: uuid
  artifact_hash: string
  review_decision: accept | needs_revision | reject
  compensation_policy_version_id: uuid
  compensation_award_ids: list[uuid]
  recorded_at: timestamp

```

Consumers may use this event for reputation signals, analytics, notifications, evidence export, or other authorized projections.

The event does not assert that compensation has been fulfilled.

14.4 CompensationAwardCreated

```

event_type: CompensationAwardCreated
event_version: 1
idempotency_key: "compensation-award:{compensation_award_id}:created:v1"

payload:
  compensation_award_id: uuid
  contribution_record_id: uuid
  project_id: uuid
  contributor_id: uuid
  instrument_type: money | project_points
  unit_code: string
  quantity: decimal_string
  compensation_policy_version_id: uuid
  adapter_binding_id: uuid
  created_at: timestamp

```

This event is an immutable domain fact. It is not itself the adapter instruction.

14.5 CompensationFulfillmentRequested

One event is emitted per CompensationAward.

```

event_type: CompensationFulfillmentRequested
event_version: 1
idempotency_key: "compensation-award:{compensation_award_id}:fulfill:v1"

payload:
  compensation_award_id: uuid
  contribution_record_id: uuid
  project_id: uuid
  contributor_id: uuid

  instrument_type: money | project_points
  unit_code: string
  quantity: decimal_string

  compensation_policy_version_id: uuid
  adapter_binding_id: uuid
  adapter_route_key: string

  contribution_type: accepted_submission | completed_review
  source_review_id: uuid
  created_at: timestamp

```

The event contains the complete authorized instruction. The adapter **MUST NOT** query mutable project policy to determine a different quantity.

14.6 CompensationFulfillmentRecorded

One event is emitted for every accepted fulfillment receipt.

```

event_type: CompensationFulfillmentRecorded
event_version: 1
idempotency_key: "fulfillment-receipt:{fulfillment_receipt_id}:recorded:v1"

payload:
  fulfillment_receipt_id: uuid
  compensation_award_id: uuid
  contribution_record_id: uuid
  project_id: uuid
  contributor_id: uuid
  instrument_type: money | project_points
  unit_code: string
  authorized_quantity: decimal_string
  reported_status: fulfilled | failed
  external_reference: string | null
  failure_code: string | null
  fulfilled_at: timestamp | null
  received_at: timestamp

```

This event reports Workstream's accepted receipt. It does not expose provider credentials or create a new award.

14.7 Sensitive-data restriction

Events **MUST NOT** contain:

- bank details;
- wallet private keys;
- provider access tokens;
- Identity Issuer bearer tokens;
- point-account credentials;
- raw personal financial data.

The adapter resolves its own external beneficiary mapping from `contributor_id` through its authorized integration.

15. External Adapter Contract

15.1 Workstream adapter port

Workstream exposes one outbound port to its outbox dispatcher. Provider-specific implementations remain outside the Workstream domain.

```
class CompensationAdapterPort(Protocol):
    async def deliver(
        self,
        binding_id: UUID,
        event: CompensationFulfillmentRequested,
    ) -> AdapterDeliveryAcknowledgement:
        ...

class AdapterDeliveryAcknowledgement:
    event_id: UUID
    accepted: Literal[True]
    acknowledged_at: datetime
```

The port implementation MUST:

- route only through the persisted adapter_binding_id;
- send the persisted event without recalculating it;
- authenticate Workstream to the adapter;
- return success only after the adapter durably stores the event idempotency key;
- translate timeouts and non-success responses into a delivery failure so the outbox retries.

The repository includes only the port, dispatcher integration, and a deterministic test adapter for the live drill. A provider-specific money or points implementation is not part of WS-CON-001.

15.2 Delivery

The outbox dispatcher routes CompensationFulfillmentRequested using the frozen adapter_binding_id and route_key.

Adapter response to delivery:

- 2xx means the adapter durably accepted the instruction and its idempotency key;
- non-2xx or timeout means delivery was not acknowledged and Workstream retries;
- a 2xx acknowledgement does not mean money or points were fulfilled.

After acknowledgement, Workstream does not create provider retries. The adapter owns all provider-side processing.

15.3 Adapter idempotency

The adapter MUST treat compensation_award_id and the event idempotency key as stable identifiers.

Repeated delivery of the same event:

- must not create another payment request;
- must not credit points twice;
- must return a successful acknowledgement after the original instruction is durably recognized.

15.4 Fulfillment callback request

```

CompensationFulfillmentReported:
  external_event_id: string
  compensation_award_id: uuid
  status: fulfilled | failed

  external_reference: string | null
  fulfilled_quantity: decimal_string | null
  fulfilled_at: timestamp | null

  failure_code: string | null
  failure_message: string | null
  reported_at: timestamp

```

The callback does not accept `project_id`, `contributor_id`, `instrument_type`, `unit_code`, or `authorized quantity` as mutable authority. Workstream loads those from `CompensationAward`.

15.5 Callback authentication and authorization

Workstream MUST:

1. verify the Identity Issuer service token;
2. resolve the adapter actor;
3. load the `CompensationAward` and frozen adapter binding;
4. require the token actor to equal the binding `adapter_actor_id`;
5. require the binding project and instrument to match the award;
6. validate callback schema and timestamps;
7. enforce idempotency before insertion.

15.6 Fulfilled callback

Validation:

- `external_reference` is non-empty;
- `fulfilled_quantity` exactly equals `award.quantity`;
- `fulfilled_at` is present and not unreasonably in the future;
- no different fulfilled receipt already exists;
- award project and binding remain internally consistent.

Effect in one transaction:

```

insert immutable fulfilled receipt
update status projection -> fulfilled
write CompensationFulfilled audit event
write CompensationFulfillmentRecorded outbox event
commit

```

Fulfilled is terminal.

15.7 Failed callback

Validation:

- `failure_code` is non-empty;
- `fulfilled_quantity` and `fulfilled_at` are null;
- no fulfilled receipt already exists.

Effect:

```
insert immutable failed receipt
update status projection -> failed
write CompensationFulfillmentFailed audit event
write CompensationFulfillmentRecorded outbox event
commit
```

The adapter may later report fulfilled using a new `external_event_id`.

15.8 Prohibited partial fulfillment

If `fulfilled_quantity` differs from `award.quantity`:

- reject with 422 `partial_fulfillment_not_supported`;
- create no receipt;
- do not change the projection;
- write a rejected-callback security/audit event.

The adapter must resolve partial provider behaviour internally before reporting the Workstream award as fulfilled.

15.9 Contradictory callbacks

- failed followed by fulfilled: allowed.
- failed followed by another failed with a new `external_event_id`: allowed.
- fulfilled followed by failed: rejected with 409 `award_already_fulfilled`.
- fulfilled followed by a different fulfilled event: rejected with 409 `award_already_fulfilled`.
- exact replay of an accepted callback: return the existing receipt with 200.
- reuse of `external_event_id` with different payload: reject with 409 `idempotency_mismatch`.

16. Reputation Boundary

`ContributionRecorded` is the canonical reputation input from this lifecycle.

Recommended raw signal mapping:

contribution_type	reputation signal
accepted_submission	submission_accepted
completed_review	review_completed

Normative rules:

- Signal creation is independent of compensation awards.
- Signal creation is independent of adapter delivery and fulfillment.
- A completed-review signal may carry `Review.decision` as provenance, but this specification assigns no positive or negative score based on that outcome.
- Needs revision does not automatically reduce submitter reputation.
- Reject does not create a submitter Contribution Record and therefore produces no positive submitter contribution signal.
- Lease expiry and release signals remain the separate audit events defined by WS-REV-001.

- Reputation scoring, aggregation, decay, project-to-global weighting, grant automation, and reviewer-quality adjudication are outside this specification.

The reputation consumer MUST deduplicate by `contribution_record_id` and event version.

17. Read Model and API Composition

17.1 Canonical versus composed data

The canonical Contribution Record remains unchanged after creation. Read APIs compose:

```
ContributionRecord
+ CompensationAwards
+ CompensationStatusProjection
+ latest immutable FulfillmentReceipts
= Contribution detail response
```

The response may display pending, failed, or fulfilled compensation without storing those fields on ContributionRecord.

17.2 Example contribution response

```
{
  "id": "contribution-uuid",
  "project_id": "project-uuid",
  "task_id": "task-uuid",
  "contributor_id": "contributor-uuid",
  "contribution_type": "completed_review",
  "source_review_id": "review-uuid",
  "submission_version_id": "submission-version-uuid",
  "recorded_at": "2026-07-10T12:00:00Z",
  "compensation": [
    {
      "award_id": "money-award-uuid",
      "instrument_type": "money",
      "unit_code": "USD",
      "quantity": "1.25",
      "delivery_status": "acknowledged_by_adapter",
      "fulfillment_status": "fulfilled",
      "external_reference": "adapter-reference"
    },
    {
      "award_id": "points-award-uuid",
      "instrument_type": "project_points",
      "unit_code": "REVIEW_POINT",
      "quantity": "5",
      "delivery_status": "acknowledged_by_adapter",
      "fulfillment_status": "pending",
      "external_reference": null
    }
  ]
}
```

This is a read representation, not a mutable aggregate stored as one row.

18. API Contract

The following paths are normative for Workstream v0.1. The `/v1` prefix is the independent API contract namespace, not a Workstream v1.0 release identifier.

18.1 Compensation policies

```

POST /v1/projects/{project_id}/compensation-policies
POST /v1/projects/{project_id}/compensation-policies/{policy_id}/versions
PUT /v1/projects/{project_id}/compensation-policy-versions/{version_id}/draft
POST /v1/projects/{project_id}/compensation-policy-versions/{version_id}/publish
POST /v1/projects/{project_id}/compensation-policy-versions/{version_id}/retire
GET /v1/projects/{project_id}/compensation-policies
GET /v1/projects/{project_id}/compensation-policy-versions/{version_id}

```

Creating a new policy creates the policy identity and an empty version 1 draft. Creating a version under an existing policy copies the current published version into the next draft version. The draft remains inactive until explicitly published.

Policy creation request:

```

{
  "name": "Project default compensation"
}

```

PUT replaces the complete draft rules; it does not merge individual fields.

```

{
  "rules": [
    {
      "contribution_type": "accepted_submission",
      "compensation_mode": "compensated",
      "awards": [
        {
          "instrument_type": "money",
          "unit_code": "USD",
          "quantity": "45.00",
          "adapter_binding_id": "money-binding-uuid"
        }
      ]
    },
    {
      "contribution_type": "completed_review",
      "compensation_mode": "compensated",
      "awards": [
        {
          "instrument_type": "money",
          "unit_code": "USD",
          "quantity": "60.00",
          "adapter_binding_id": "money-binding-uuid"
        },
        {
          "instrument_type": "project_points",
          "unit_code": "REVIEW_POINT",
          "quantity": "5",
          "adapter_binding_id": "points-binding-uuid"
        }
      ]
    }
  ]
}

```

The publish request and the retire request both require:

```

{
  "expected_current_version_id": "current-version-uuid-or-null"
}

```

No edit endpoint exists for a published or retired version.

18.2 Adapter bindings

```
POST /v1/projects/{project_id}/compensation-adapter-bindings
POST /v1/projects/{project_id}/compensation-adapter-bindings/{binding_id}/suspend
POST /v1/projects/{project_id}/compensation-adapter-bindings/{binding_id}/resume
POST /v1/projects/{project_id}/compensation-adapter-bindings/{binding_id}/retire
GET /v1/projects/{project_id}/compensation-adapter-bindings
```

Binding creation request:

```
{
  "instrument_type": "money",
  "adapter_actor_id": "service-actor-uuid",
  "route_key": "project-money-v1"
}
```

The route_key identifies deployment routing configuration and contains no endpoint credential or secret.

18.3 Contributions

```
GET /v1/contributions/{contribution_record_id}
GET /v1/contributors/me/contributions
GET /v1/contributors/{contributor_id}/contributions
GET /v1/projects/{project_id}/contributions
GET /v1/projects/{project_id}/tasks/{task_id}/contributions
```

Supported filters:

- project_id;
- contribution_type;
- task_id;
- recorded_from;
- recorded_to;
- cursor.

Ordering is deterministic:

```
recorded_at DESC, id DESC
```

Authorization is deterministic:

- a contributor may use contributors/me for their complete authorized history;
- an Operator may use contributors/{contributor_id} across projects;
- every other Admin using contributors/{contributor_id} MUST supply project_id and hold authorized Admin scope for that project;
- project and task endpoints enforce their path project scope;
- records from unauthorized projects are never returned and are not revealed through total counts.

No POST, PATCH, PUT, or DELETE Contribution Record endpoint exists.

18.4 Compensation awards

```
GET /v1/compensation-awards/{award_id}
GET /v1/projects/{project_id}/compensation-awards
GET /v1/contributors/{contributor_id}/compensation-awards
```

Supported operational filters:

- instrument_type;
- delivery_status;
- fulfillment_status;
- created_from;
- created_to;
- minimum_pending_age.

18.5 Fulfillment callback

POST /v1/integrations/compensation/fulfillment-reports

Responses:

- 201 when a new receipt is created;
- 200 for exact idempotent replay;
- 400 for malformed schema;
- 401 for invalid identity;
- 403 for unauthorized adapter or project/instrument mismatch;
- 404 for unknown award;
- 409 for callback conflict or idempotency mismatch;
- 422 for invalid fulfillment semantics.

No endpoint allows an adapter to change the Contribution Record, CompensationAward, policy, assignment, lease, Review, task, or contributor.

19. Error Contract

Errors MUST use the existing Workstream structured-error envelope and stable machine codes.

Code	HTTP	Meaning
compensation_policy_missing	409	Project has no active published compensation version
compensation_policy_not_draft	409	Attempt to edit non-draft version
compensation_policy_invalid	422	Draft violates policy rules
compensation_policy_integrity_error	500	Frozen reference or published data is internally inconsistent
compensation_authority_required	403	Actor lacks required Admin authority
adapter_binding_missing	409	Required instrument has no compatible binding
adapter_binding_in_use	409	Binding is referenced by active policy, unfinished frozen work, or unfulfilled awards
adapter_unauthorized	403	Callback actor is not bound to the award
contribution_not_found	404	Contribution Record does not exist or is not visible
compensation_award_not_found	404	Award does not exist or is not visible
contribution_already_recorded	409	Logical contribution already exists
compensation_award_already_exists	409	Logical award already exists
fulfillment_callback_invalid	422	Callback fields do not match status rules

Code	HTTP	Meaning
partial_fulfillment_not_supported	422	Fulfilled quantity differs from authorized quantity
award_already_fulfilled	409	A contradictory post-fulfillment callback was attempted
idempotency_mismatch	409	Idempotency identifier was reused with different payload

Internal uniqueness races MUST be translated into the corresponding stable conflict or idempotent success rather than leaking database errors.

20. Database Constraints and Indexes

Required constraints:

```

ContributionRecord:
  UNIQUE(source_review_id, contribution_type)
  CHECK(contribution_type IN accepted_submission, completed_review)

CompensationPolicyVersion:
  UNIQUE(compensation_policy_id, version_number)

CompensationPolicy:
  partial UNIQUE(project_id) WHERE status = active

CompensationRule:
  UNIQUE(compensation_policy_version_id, contribution_type)
  CHECK(compensation_mode IN compensated, unpaid)

CompensationAwardDefinition:
  UNIQUE(compensation_policy_version_id, contribution_type, instrument_type)
  CHECK(quantity > 0)

CompensationAward:
  UNIQUE(contribution_record_id, instrument_type)
  CHECK(quantity > 0)

ProjectCompensationAdapterBinding:
  partial UNIQUE(project_id, instrument_type) WHERE status = active

CompensationFulfillmentReceipt:
  UNIQUE(adapter_binding_id, external_event_id)
  partial UNIQUE(compensation_award_id) WHERE reported_status = fulfilled

CompensationStatusProjection:
  PRIMARY KEY(compensation_award_id)

TaskAssignment:
  submitter_compensation_policy_version_id NOT NULL

ReviewLease:
  reviewer_compensation_policy_version_id NOT NULL

```

Foreign keys MUST prevent project-chain mismatches. Composite foreign keys MUST be used where a single-column foreign key cannot express project ownership. Transaction-level validation remains additional defence and MUST NOT replace enforceable database ownership constraints.

Required indexes:

```

ContributionRecord(contributor_id, recorded_at DESC, id DESC)
ContributionRecord(project_id, recorded_at DESC, id DESC)
ContributionRecord(project_id, task_id, recorded_at, id)
ContributionRecord(source_review_id, contribution_type)

CompensationPolicyVersion(project_id, status)
CompensationAward(contribution_record_id)
CompensationAward(project_id, created_at DESC)
CompensationAward(adapter_binding_id, created_at)

CompensationFulfillmentReceipt(compensation_award_id, received_at)
CompensationStatusProjection(fulfillment_status, updated_at)

Outbox(event_type, delivery_state, next_attempt_at)

```

All timestamps are stored in UTC.

21. Concurrency and Idempotency

21.1 Review-decision replay

The WS-REV-001 decision idempotency key protects the entire combined transaction.

Repeating the same committed decision request **MUST** return:

- the existing Review;
- the existing reviewer Contribution Record;
- the existing submitter Contribution Record when decision = accept;
- the existing CompensationAwards;
- no new outbox events.

Reusing the decision idempotency key with a different decision or payload returns 409 idempotency_mismatch.

21.2 Concurrent decision attempts

If two requests attempt to decide the same ReviewLease or SubmissionVersion:

- exactly one may create the Review;
- exactly one completed_review Contribution Record exists;
- at most one accepted_submission Contribution Record exists;
- award uniqueness follows the winning contribution records;
- the losing transaction returns the existing result only if it is an exact idempotent replay; otherwise it returns a conflict.

21.3 Concurrent policy activation

Project policy activation locks the project active-version reference.

If two versions are activated concurrently:

- exactly one becomes active first;
- the other transaction must re-evaluate and either replace it explicitly under its request semantics or fail with a version conflict;
- assignments and leases freeze one committed version, never a mixture.

Activation endpoints MUST require `expected_current_version_id`. Null is supplied only when the project has no active version.

21.4 Policy activation versus assignment

Assignment and lease creation lock the same project active-version reference used by activation.

The result is serializable at that boundary:

- work freezes the old version before activation commits; or
- work freezes the new version after activation commits.

Server request timing outside the transaction does not determine the version.

21.5 Award duplication

CompensationAward creation is part of the contribution transaction and protected by uniqueness constraints.

An insert conflict caused by an exact replay resolves to the existing award. A conflict with different unit, quantity, policy version, or binding is an integrity error.

21.6 Outbox delivery replay

Outbox delivery is at-least-once.

- The `event_id` and `idempotency_key` remain unchanged across retries.
- Delivery-attempt count and next-attempt time are operational metadata.
- A transient adapter failure does not create another CompensationAward or outbox event.
- After durable adapter acknowledgement, provider-side retries belong to the adapter.

21.7 Concurrent callbacks

Callback processing locks the CompensationAward or its status row.

If failed and fulfilled callbacks race:

- if fulfilled commits first, the failed callback is rejected;
- if failed commits first, the fulfilled callback may subsequently commit;
- final state is fulfilled.

If two different fulfilled callbacks race, exactly one may create the unique fulfilled receipt. The other is rejected unless it is the exact idempotent replay of the winner.

21.8 Callback before delivery acknowledgement

A valid authenticated callback may be received before Workstream records local delivery acknowledgement.

In that case:

- accept and store the receipt;
- set `fulfillment_status` according to the receipt;
- set `delivery_status` to `acknowledged_by_adapter` because possession of the award instruction is proven;
- mark the associated outbox instruction delivered or suppress further delivery idempotently;
- never regress fulfilled state when a late dispatcher acknowledgement arrives.

22. Background Processing and Recovery

22.1 Outbox dispatcher

The dispatcher:

- selects committed undelivered events;
- routes by `adapter_binding_id` and `route_key`;
- sends the exact persisted payload;
- records delivery attempts;
- retries transient failures using $\text{delay} = \min(300 \text{ seconds}, 2^{\text{attempt_number}} \text{ minus } 1 \text{ seconds})$, plus uniformly distributed jitter from zero through ten percent of that delay;
- marks acknowledged only after a valid adapter 2xx response;
- preserves events until retention policy permits archival.

It MUST NOT derive a new award quantity from current policy.

There is no maximum delivery-attempt count in v0.1. After ten consecutive failed attempts, the dispatcher continues retrying and triggers the delayed-delivery alert.

22.2 Status projection updater

Projection updates happen synchronously in the same transaction that records adapter delivery acknowledgement or a fulfillment receipt. Projection rebuild remains an asynchronous recovery operation.

The updater:

- updates are idempotent;
- fulfilled cannot regress;
- the projection can be fully rebuilt;
- source receipts and delivery records remain authoritative.

22.3 Reconciliation job

The reconciliation job detects:

- Review without required reviewer Contribution Record;
- accept Review without submitter Contribution Record;
- Contribution Record without ContributionRecorded outbox event;
- Contribution Record whose frozen policy version does not match its assignment or lease;
- award missing for a published definition;
- unexpected award for an unpaid contribution type;
- award without CompensationFulfillmentRequested;
- fulfilled projection without fulfilled receipt;
- fulfilled receipt whose quantity differs from award;
- multiple fulfilled receipts;
- adapter binding retired while referenced by an unfulfilled award;
- acknowledged delivery whose adapter binding does not match the award.

Reconciliation **MUST** alert operators and use explicit replay or compensating operational actions. It **MUST NOT** silently edit immutable Reviews, Contribution Records, awards, definitions, or receipts.

22.4 Event replay

Safe replay is allowed for:

- rebuilding read projections;
- recreating a missing delivery attempt from an existing outbox event;
- re-delivering an unacknowledged event with the same identifiers;
- recreating a missing mutable status projection from authoritative records.

Replay **MUST NOT** create a second logical Contribution Record, award, or fulfilled receipt.

22.5 Adapter suspension

When a binding is suspended:

- existing awards remain valid;
- undelivered events remain pending;
- no award is cancelled;
- Review decisions and contribution creation continue if the policy version was already frozen;
- new policy publication cannot select the suspended binding;
- new TaskAssignments or ReviewLeases cannot freeze a current policy that depends on a suspended binding.

Existing assignments and leases are honored because contributors began work under frozen terms. Delivery resumes when the same logical binding is reactivated.

23. Audit Events

The following event types are required.

Policy

- CompensationPolicyCreated
- CompensationPolicyVersionDrafted
- CompensationPolicyVersionPublished
- CompensationPolicyVersionActivated
- CompensationPolicyVersionRetired
- CompensationAwardDefinitionCreated
- ProjectCompensationAdapterBound
- ProjectCompensationAdapterSuspended
- ProjectCompensationAdapterResumed
- ProjectCompensationAdapterRetired

Frozen terms

- CompensationTermsFrozenForAssignment

- CompensationTermsFrozenForReviewLease

Contribution

- ReviewerContributionRecorded
- SubmitterContributionRecorded
- ContributionRecorded

Award and delivery

- CompensationAwardCreated
- CompensationFulfillmentRequested
- CompensationFulfillmentDeliveryAttempted
- CompensationFulfillmentDeliveryAcknowledged
- CompensationFulfillmentDeliveryFailed

Callback

- CompensationFulfillmentReported
- CompensationFulfilled
- CompensationFulfillmentFailed
- CompensationFulfillmentCallbackRejected
- CompensationFulfillmentCallbackReplayed

Every audit event records:

```
event_id: uuid
event_type: string
occurred_at: timestamp
actor_id: actor_id | system_actor_id

project_id: uuid
contributor_id: uuid | null
task_id: uuid | null
review_id: uuid | null
review_lease_id: uuid | null
contribution_record_id: uuid | null
compensation_award_id: uuid | null
compensation_policy_version_id: uuid | null
adapter_binding_id: uuid | null

correlation_id: string
causation_event_id: uuid | null
reason_code: string | null
metadata: object
```

Audit metadata MUST redact credentials and sensitive financial data.

24. Notifications and Operational Views

Notification transport is an adapter concern. Workstream emits events sufficient to notify:

- contributor that a submission contribution was recorded;
- reviewer that a review contribution was recorded;
- contributor of the compensation instruments authorized;

- Admin that adapter delivery is delayed;
- contributor or Admin that fulfillment was reported failed;
- contributor that fulfillment was reported completed;
- Admin that policy or adapter binding is missing or suspended.

24.1 Contributor view

Must show:

- global contribution history;
- project filter;
- contribution type and source task;
- recorded time;
- compensation awarded per instrument;
- pending, failed, or fulfilled status from the projection;
- external reference only when permitted by data policy.

It MUST NOT imply that adapter acknowledgement means fulfillment.

24.2 Project Admin view

Must show:

- active compensation version;
- historical frozen versions;
- assignment and lease counts by frozen version;
- contribution counts by type;
- awards by instrument and unit;
- oldest pending-delivery age;
- oldest pending-fulfillment age;
- failed fulfillment count and failure codes;
- adapter binding health and suspension state.

24.3 Finance Authority view

Within authorized project scope, Finance Authority may read:

- monetary award totals;
- monetary awards by fulfillment status;
- policy versions containing money;
- money-adapter delivery health;
- external references returned by the authorized adapter.

Workstream does not provide provider balances, payout batches, or settlement ledgers.

24.4 Reputation Authority view

Within authorized project scope, Reputation Authority may read:

- Contribution Records;

- contribution types;
- source Reviews and provenance;
- ContributionRecorded event delivery status to the reputation consumer.

Compensation fulfillment is not a reputation input.

25. Observability

Required metrics:

```
workstream_contributions_created_total{project_id, contribution_type}
workstream_contribution_transaction_failures_total{reason}
workstream_compensation_awards_created_total{project_id, instrument_type, unit_code}
workstream_compensation_award_quantity_total{project_id, instrument_type, unit_code}
workstream_compensation_outbox_pending_total{adapter_binding_id}
workstream_compensation_outbox_oldest_seconds{adapter_binding_id}
workstream_compensation_delivery_attempts_total{adapter_binding_id, result}
workstream_compensation_fulfillment_callbacks_total{adapter_binding_id, status, result}
workstream_compensation_pending_fulfillment_total{project_id, instrument_type}
workstream_compensation_pending_fulfillment_oldest_seconds{project_id, instrument_type}
workstream_compensation_reconciliation_violations_total{type}
```

Required tracing attributes:

- correlation_id;
- project_id;
- task_id;
- review_id;
- contribution_record_id;
- compensation_award_id;
- adapter_binding_id;
- event_id.

Logs MUST NOT contain bearer tokens, credentials, private keys, bank details, or full provider payloads containing sensitive data.

Suggested alerts:

- outbox oldest age exceeds project operational threshold;
- adapter delivery failure rate exceeds threshold;
- fulfillment callback rejection spikes;
- reconciliation detects any missing contribution or duplicate fulfilled receipt;
- an active policy loses a usable adapter binding;
- monetary awards remain pending beyond project service target.

26. Security Requirements

1. Verify Identity Issuer tokens on every API and callback.
2. Authorize project scope after identity verification.
3. Require elevated authority for policy publication and adapter binding.
4. Require Finance Authority or Operator for any policy containing money.

5. Never allow contributors to create, edit, fulfill, or delete their own Contribution Records or awards.
6. Bind callbacks to the exact adapter actor, project, instrument, and award.
7. Treat callback payloads as untrusted input.
8. Use exact decimal parsing with bounded length and precision.
9. Reject reported_at or fulfilled_at more than five minutes ahead of database time. A fulfilled_at value must not be more than five minutes later than reported_at.
10. Enforce request-body size limits.
11. Rate-limit callback endpoints per adapter actor.
12. Store adapter credentials outside domain tables.
13. Sign or mutually authenticate adapter delivery in addition to service identity where deployment policy requires it.
14. Prevent insecure direct-object references in global contribution and award endpoints.
15. Redact external failure messages before showing them to contributors.
16. Preserve immutable audit evidence for policy publication, award creation, delivery, and callback validation.
17. Do not expose artifact contents through contribution-list endpoints; expose only authorized references or hashes.
18. Treat money and project-points fulfillment events as economically sensitive messages.

27. End-to-End Reference Sequences

27.1 Accepted submission with paid reviewer and paid submitter

1. TaskAssignment created under CompensationPolicyVersion P1
 - submitter terms frozen to P1
2. Submission v1 passes checkers and enters review queue
3. Reviewer claims under current policy P1
 - ReviewLease terms frozen to P1
4. Reviewer records accept
5. One transaction:
 - Review accept created
 - lease consumed and queue closed
 - Task.status -> accepted
 - TaskAssignment.status -> completed
 - reviewer completed_review ContributionRecord created
 - reviewer awards created from P1 completed_review rules
 - submitter accepted_submission ContributionRecord created
 - submitter awards created from P1 accepted_submission rules
 - outbox events created
 - commit
6. Adapter deliveries occur after commit
7. Money and points adapters acknowledge independently
8. Each adapter later reports fulfilled or failed
9. Workstream stores receipts and updates read projections

27.2 Needs revision with reviewer compensation

1. Reviewer records needs_revision with required findings
2. One transaction:
 - Review created
 - task -> needs_revision
 - reviewer completed_review ContributionRecord created
 - reviewer awards created independent of decision
 - no submitter Contribution Record
 - commit
3. Contributor submits v2
4. v2 is reviewed later
5. That later valid Review creates another reviewer Contribution Record

27.3 Reject with reviewer compensation

1. Reviewer records reject
2. One transaction:
 - Review reject created
 - reviewer completed_review ContributionRecord and awards created
 - submitter TaskAssignment blocked
 - task closed
 - no submitter Contribution Record
 - commit
3. Reviewer compensation proceeds through external adapter

27.4 Explicitly unpaid project

1. Project has active published unpaid policy
2. Reviewer records accept
3. Reviewer and submitter Contribution Records are created
4. No CompensationAwards are created
5. ContributionRecorded events still feed reputation and history

27.5 Money plus project points

```

completed_review ContributionRecord
  -> money award USD 1.25 -> money adapter
  -> project points award REVIEW_POINT 5 -> points adapter

money callback -> fulfilled
points callback -> failed

read view:
  money = fulfilled
  points = failed
  contribution remains permanently recorded

```

27.6 Adapter failure followed by success

1. Adapter acknowledges award instruction
2. Adapter reports failed with external_event_id F1
3. Workstream stores failed receipt; projection -> failed
4. Adapter retries provider work internally
5. Adapter reports fulfilled with external_event_id F2
6. Workstream stores fulfilled receipt; projection -> fulfilled
7. A later failed callback is rejected

27.7 Policy changes during existing work

1. TaskAssignment freezes P1
2. Reviewer R1 claims and freezes P1
3. Project activates P2
4. R1 completes review:
 - reviewer award uses P1
 - accepted submitter award uses TaskAssignment P1
5. Reviewer R2 claims later:
 - R2 lease freezes P2
6. No existing object is rewritten

27.8 Duplicate decision and duplicate callback

1. Decision request commits Review, contributions, awards, and events
2. Same idempotency key and payload is retried
3. Existing complete result is returned; no duplicate records
4. Adapter callback commits one receipt
5. Same external_event_id and payload is retried
6. Existing receipt is returned with 200

28. Conformance Tests

An implementation is not conformant until all applicable tests pass against PostgreSQL and the real API transaction boundary.

28.1 Policy tests

1. Project without an active policy cannot create a TaskAssignment.
2. Project without an active policy cannot create a ReviewLease.
3. Explicit unpaid policy allows work and creates no awards.
4. Draft policy may be edited by authorized actor.
5. Published policy cannot be edited.
6. Retired policy cannot be edited.
7. Money policy cannot be published by Project Manager without Finance Authority.
8. Points-only policy can be published by Project Manager.
9. Duplicate money definition for one contribution type is rejected.
10. Duplicate points definition for one contribution type is rejected.
11. Zero or negative quantity is rejected.
12. Floating-point JSON quantity is rejected when the API requires a decimal string.
13. Invalid currency code is rejected.
14. Fractional project points are rejected.
15. Definition using another project's binding is rejected.
16. Dynamic outcome, contributor, task, skill, tier, or reputation condition is rejected.
17. Concurrent activation produces one deterministic active version.

28.2 Freeze tests

18. TaskAssignment stores the active version at creation.
19. ReviewLease stores the active version at claim.

20. Later activation does not change existing assignment.
21. Later activation does not change existing lease.
22. New assignment after activation uses the new version.
23. New lease after activation uses the new version.
24. Retired frozen version remains usable at decision time.
25. Suspended binding prevents new work from freezing a dependent current policy.
26. Existing frozen work remains recognizable while binding is suspended.

28.3 Contribution tests

27. Accept creates exactly one reviewer contribution.
28. Accept creates exactly one submitter contribution.
29. Needs revision creates exactly one reviewer contribution and no submitter contribution.
30. Reject creates exactly one reviewer contribution and no submitter contribution.
31. Review decision does not change reviewer award quantity.
32. Second completed revision review creates a second reviewer contribution.
33. Released lease creates no contribution.
34. Expired lease creates no contribution.
35. Revoked lease creates no contribution.
36. Automated checker result creates no Contribution Record.
37. Contribution project, task, assignment, submission, Review, and contributor chain must match.
38. Contribution Record cannot be updated.
39. Contribution Record cannot be deleted.
40. Global contributor view returns records from authorized projects.
41. Project filter returns only that project.

28.4 Atomicity and idempotency tests

42. Failure creating reviewer contribution rolls back Review.
43. Failure creating submitter contribution rolls back accept Review.
44. Failure creating an award rolls back Review and all contributions.
45. Failure writing an outbox event rolls back Review, contributions, and awards.
46. External adapter outage does not roll back committed Review.
47. Exact decision replay returns existing Review, contributions, and awards.
48. Changed payload with reused decision idempotency key is rejected.
49. Concurrent decision attempts create one Review and one logical set of contributions.
50. Award uniqueness prevents duplicates under replay.

28.5 Compensation evaluation tests

51. Completed-review money award is copied exactly from frozen definition.
52. Completed-review points award is copied exactly from frozen definition.
53. Accepted-submission awards use TaskAssignment frozen version.

- 54. Reviewer awards use ReviewLease frozen version.
- 55. One contribution can create money and points awards independently.
- 56. Unpaid contribution creates zero awards.
- 57. Current project policy is never consulted instead of frozen reference.
- 58. Missing frozen version fails with integrity error and commits nothing.
- 59. Money award carries project_id and project binding.
- 60. Same points unit text in different projects remains separately scoped.

28.6 Delivery tests

- 61. CompensationFulfillmentRequested is committed with each award.
- 62. Dispatcher sends exact persisted payload.
- 63. Retry preserves event_id and idempotency_key.
- 64. Non-2xx adapter response leaves event unacknowledged.
- 65. 2xx acknowledgement marks delivery acknowledged but fulfillment pending.
- 66. Repeated delivery does not create another award.
- 67. Suspended binding pauses delivery without cancelling award.

28.7 Callback tests

- 68. Bound adapter may report its own award.
- 69. Invalid Identity Issuer token is rejected.
- 70. Money adapter cannot report points award.
- 71. Project A adapter cannot report Project B award.
- 72. Unknown award is rejected.
- 73. Fulfilled callback requires external reference, quantity, and timestamp.
- 74. Fulfilled quantity must exactly equal award quantity.
- 75. Partial fulfillment is rejected.
- 76. Failed callback requires failure code.
- 77. Failed callback creates immutable receipt.
- 78. Failed followed by fulfilled is allowed.
- 79. Fulfilled followed by failed is rejected.
- 80. Exact callback replay returns existing receipt.
- 81. Reused external_event_id with different payload is rejected.
- 82. Concurrent fulfilled callbacks produce one fulfilled receipt.
- 83. Callback does not mutate Contribution Record.
- 84. Callback does not mutate CompensationAward.
- 85. Valid callback before delivery acknowledgement does not regress later.

28.8 Read and recovery tests

- 86. Contribution detail composes award status without modifying canonical record.
- 87. Projection can be rebuilt from awards, delivery records, and receipts.

88. Reconciliation detects accepted Review missing submitter contribution.
89. Reconciliation detects Review missing reviewer contribution.
90. Reconciliation detects missing award.
91. Reconciliation detects unexpected award for unpaid policy.
92. Reconciliation detects missing fulfillment event.
93. Reconciliation detects projection/receipt mismatch.
94. Reconciliation never silently edits immutable records.
95. Binding retirement is rejected while an active policy, unfinished frozen work, or unfulfilled award depends on it.
96. Retiring the active policy without replacement clears the project active-version reference and blocks new work.
97. A non-Operator Admin cannot use the contributor-global endpoint without an authorized project filter.

29. Implementation Delivery Order

Phase 1: Schema and immutable policy model

- CompensationPolicy and version tables;
- CompensationRule;
- award definitions;
- adapter bindings;
- frozen references on TaskAssignment and ReviewLease;
- ContributionRecord;
- CompensationAward;
- FulfillmentReceipt;
- status projection;
- constraints and indexes.

Phase 2: Policy and binding APIs

- draft creation and validation;
- authority enforcement;
- publish/activate/retire transaction;
- adapter binding lifecycle;
- explicit unpaid policy;
- concurrency guards.

Phase 3: Freeze integration

- TaskAssignment freeze;
- ReviewLease freeze;
- policy-change concurrency tests;
- legacy-data migration guard;

- suspended-binding behaviour.

Phase 4: Atomic Review integration

- reviewer contribution for all decisions;
- submitter contribution for accept;
- deterministic award evaluation;
- outbox events;
- exact decision replay;
- rollback and race tests;
- removal of ContributionRecordRequested emission.

Phase 5: External adapter boundary

- outbox routing;
- delivery acknowledgement;
- callback authentication and authorization;
- immutable receipts;
- status projection;
- conflict and idempotency handling.

Phase 6: Read APIs and operations

- global and project-filtered contribution views;
- compensation summaries;
- Admin operational views;
- metrics, tracing, alerts;
- reconciliation and projection rebuild.

Phase 7: Live API drill

The drill MUST prove at minimum:

1. paid accept producing reviewer and submitter contributions;
2. needs revision producing reviewer contribution only;
3. reject producing reviewer contribution only;
4. explicit unpaid policy;
5. policy change with frozen old and new work;
6. money and project-points awards from one contribution;
7. adapter acknowledgement without fulfillment;
8. failed callback followed by fulfilled callback;
9. exact decision and callback replay;
10. adapter outage with eventual delivery;
11. database evidence for atomicity and uniqueness;
12. projection rebuild and reconciliation.

The drill report must clearly separate observed runtime proof from specification claims.

30. Definition of Done

WS-CON-001 is implemented only when:

- every valid Review creates exactly one reviewer Contribution Record;
 - accept additionally creates exactly one submitter Contribution Record;
 - no other outcome creates a submitter Contribution Record;
 - review compensation is identical across accept, needs_revision, and reject under the same frozen policy;
 - TaskAssignment and ReviewLease freeze immutable policy versions;
 - policy replacement cannot alter existing work;
 - explicit unpaid policy works without missing configuration;
 - Contribution Records and CompensationAwards are immutable;
 - money and project points remain project-scoped;
 - Contribution Records remain globally addressable and project-filterable;
 - Review, contribution, awards, and outbox events commit atomically;
 - adapter outage cannot invalidate a committed Review;
 - delivery acknowledgement and fulfillment are represented separately;
 - callbacks are authenticated, authorized, idempotent, and exact-quantity;
 - failed may become fulfilled, while fulfilled is terminal;
 - fulfillment receipts are immutable;
 - read projections are rebuildable;
 - provider-specific payment and points internals are absent from Workstream;
 - all conformance tests pass;
 - the live drill captures API, database, event, retry, callback, and audit evidence.
-

31. Explicitly Out of Scope

The coding agent MUST NOT implement any of the following under WS-CON-001:

- payment-request objects;
- payment-attempt objects;
- payment-provider SDK logic;
- bank, wallet, card, stablecoin, x402, or settlement execution;
- payout batching;
- balances or account statements;
- KYC or beneficiary onboarding;
- finance-approval workflow inside Workstream;
- provider retry or provider reconciliation logic;
- project-points ledger;

- point balances or transfers;
- Workstream-wide points;
- credits or credit ledger;
- token issuance;
- compensation tiers;
- task-specific compensation overrides;
- contributor-specific rates;
- skill- or reputation-derived rates;
- bonuses, penalties, multipliers, or dynamic formulas;
- partial fulfillment;
- currency conversion;
- reputation score calculation;
- reputation aggregate maintenance;
- automated reviewer-role grants;
- adjudication or second-level review;
- contribution reversal, voiding, or adjustment;
- retroactive policy application;
- mutation or deletion of canonical Contribution Records, awards, or receipts;
- unrelated product architectures.

External adapters may later connect to those systems, but this specification defines only the Workstream-facing contract.

32. Future Additive Directions

The following may be designed later through separate specifications:

- provider-specific money adapters;
- project-points service and ledger;
- credits and other compensation instruments;
- finance approval adapters;
- task-category and compensation-tier policy rules;
- bonuses or quality incentives based on independently verified evidence;
- partial fulfillment;
- compensation adjustment and reversal records;
- adjudication and audit review;
- Workstream-wide economic reporting;
- reputation scoring and project-to-global aggregation;
- automated ProjectRoleGrant policy;
- external evidence publication.

Future additions MUST preserve:

- original Contribution Records;
- original CompensationAwards;
- frozen policy references;
- original fulfillment receipts;
- decision-neutral base reviewer compensation;
- project scope;
- event idempotency.

33. Coding-Agent Handoff Rules

The coding agent MUST follow these rules without reinterpretation:

1. Implement this specification as an extension of WS-REV-001, not as a separate competing lifecycle.
2. Replace the temporary ContributionRecordRequested seam with direct atomic creation.
3. Keep the combined Review transaction short and database-local.
4. Do not call external adapters inside the Review transaction.
5. Do not use current project policy when a frozen assignment or lease version exists.
6. Do not derive reviewer compensation from Review.decision.
7. Do not create submitter contribution for needs_revision or reject.
8. Do not create contribution for an incomplete lease.
9. Do not store fulfillment state on ContributionRecord or CompensationAward.
10. Do not build payment, point-ledger, credit, or provider domain models.
11. Use exact decimal types and decimal-string API fields.
12. Enforce uniqueness in PostgreSQL, not only application code.
13. Require idempotency for Review decisions, event consumers, adapter delivery, and callbacks.
14. Treat external adapter payloads as untrusted.
15. Preserve project scope on every economic record and event.
16. Keep global contribution views as queries/read models, not a second global ledger.
17. Use append-only receipts and rebuildable projections.
18. Never silently repair immutable history.
19. Add no adjudication, appeal, reversal, or adjustment path.
20. Add no conditional compensation rule beyond exact contribution_type matching.
21. If existing code conflicts with a MUST rule, stop and raise the conflict before weakening the specification.
22. Completion requires conformance tests and a live API drill report; code compilation alone is insufficient.

34. Final Implementation Contract

The complete v0.1 contract is:

Workstream records every valid completed Review as one immutable reviewer contribution, regardless of decision. An accepted Review additionally records one immutable submitter contribution. Each contribution is evaluated against compensation terms frozen before work began. Workstream creates immutable, project-scoped money or project-points awards and emits transactional fulfillment instructions. External adapters perform all provider and ledger work and report immutable fulfilled or failed results. Contribution and award history never changes; API views compose current fulfillment status from append-only evidence and rebuildable projections.